

hw1

Yi-Ting Tsai

This is the link to my repo: <https://github.com/ytting920410/Homework1>

Import library

```
library(readODS)
```

Warning: package 'readODS' was built under R version 4.5.3

```
library(roxygen2)
```

Warning: package 'roxygen2' was built under R version 4.5.3

Problem1:

a.

```
soybean <- read.csv('data/data.csv', header = TRUE)
class(soybean)
```

```
[1] "data.frame"
```

```
head(soybean)
```

	Season	Cultivar	Repetition	PH	IFP	NLP	NGP	NGL	NS	MHG	GY
1	1	NEO 760 CE	1	58.80	15.20	98.2	177.80	1.81	5.2	152.20	3232.82
2	1	NEO 760 CE	2	58.60	13.40	102.0	195.00	1.85	7.2	141.69	3517.36
3	1	NEO 760 CE	3	63.40	17.20	100.4	203.00	2.02	6.8	148.81	3391.46
4	1	NEO 760 CE	4	60.27	15.27	100.2	191.93	1.89	6.4	148.50	3312.58
5	1	MANU IPRO	1	81.20	18.00	98.8	173.00	1.75	7.4	145.59	3230.99
6	1	MANU IPRO	2	75.80	20.80	69.2	128.00	1.85	7.2	154.87	3374.80

```
summary(soybean)
```

Season		Cultivar	Repetition	PH			
Min.	:1.0	Length:320	Min.	:1.00	Min.	:47.60	
1st Qu.:	1.0	Class :character	1st Qu.:	1.75	1st Qu.:	62.95	
Median	:1.5	Mode :character	Median	:2.50	Median	:67.20	
Mean	:1.5		Mean	:2.50	Mean	:68.39	
3rd Qu.:	2.0		3rd Qu.:	3.25	3rd Qu.:	74.35	
Max.	:2.0		Max.	:4.00	Max.	:94.80	
IFP		NLP	NGP	NGL			
Min.	: 7.20	Min.	: 20.20	Min.	: 47.80	Min.	: 0.940
1st Qu.:	13.60	1st Qu.:	44.35	1st Qu.:	95.05	1st Qu.:	2.000
Median	:15.60	Median	: 54.50	Median	:123.00	Median	: 2.280
Mean	:15.46	Mean	: 59.09	Mean	:135.09	Mean	: 2.291
3rd Qu.:	17.33	3rd Qu.:	71.22	3rd Qu.:	161.35	3rd Qu.:	2.480
Max.	:26.40	Max.	:123.00	Max.	:683.40	Max.	:14.860
NS		MHG	GY				
Min.	:0.400	Min.	:127.1	Min.	:1538		
1st Qu.:	3.000	1st Qu.:	153.8	1st Qu.:	3127		
Median	:3.800	Median	:166.2	Median	:3397		
Mean	:4.072	Mean	:168.3	Mean	:3419		
3rd Qu.:	5.000	3rd Qu.:	183.2	3rd Qu.:	3708		
Max.	:9.000	Max.	:216.0	Max.	:4930		

```
colSums(is.na(soybean))
```

Season	Cultivar	Repetition	PH	IFP	NLP	NGP
0	0	0	0	0	0	0
NGL	NS	MHG	GY			
0	0	0	0			

No NA in this dataset.

Adjust the data

```
soybean$Season <- factor(soybean$Season)
soybean$Cultivar <- factor(soybean$Cultivar)
# Season and Cultivar should be factor

soybean$PH <- as.numeric(soybean$PH)
soybean$GY <- as.numeric(soybean$GY)
```

```
soybean$MHG <- as.numeric(soybean$MHG)
# PH, GY, and MHG should be numeric
```

b.

```
nrow(soybean) # Number of observations
```

```
[1] 320
```

```
length(unique(soybean$Cultivar)) # Number of distinct cultivars
```

```
[1] 40
```

```
table_seasonal <- table(soybean$Season)
# Showing the number of observation in each season
table_seasonal
```

```
  1   2
160 160
```

There are 320 observations in the dataset and 40 distinct cultivars. There are 160 observations for each season.

c.

```
cor(soybean$PH, soybean$GY, method = 'pearson')
```

```
[1] 0.1232807
```

```
# 1. correlation between PH and GY

cor_by_season <- sapply(
  split(soybean, soybean$Season),
  function(x) cor(x$PH, x$GY, method = 'pearson')
)
cor_by_season # 2. correlation between PH and GY for each season
```

```
      1      2
0.2308581 -0.0227046
```

```
tallest <- soybean[soybean$PH == max(soybean$PH), ]
tallest$GY # 3. Find GY of the tallest observation
```

```
[1] 3304.678
```

```
mean(soybean$MHG > mean(soybean$MHG)) * 100
```

```
[1] 47.1875
```

```
# 4. Percentage of observations have MHG greater than average

cul_highGY <- aggregate(
  GY ~ Cultivar,
  data = soybean,
  FUN = function(x) mean(x)
)
cul_highGY[order(cul_highGY$GY, decreasing = TRUE), ][1, ]
```

```
  Cultivar      GY
9 98R30 CE 3964.968
```

```
# 5. cultivar with the highest average GY
```

1. The correlation is 0.1232807.
2. Season 1 has the higher correlation.
3. The grain yield of the observation with the greatest plant height is 3304.678.
4. The percentage of observations with MHG greater than overall average MHG is 47.1875.
5. Cultivar 98R30 CE has the highest average grain yield with 3964.968.

d.

```
overall_means <- data.frame( # overall averages
  Group = 'overall',
  as.list(
    sapply(
      soybean[c('PH', 'IFP', 'NLP', 'NGP', 'NGL', 'NS', 'MHG', 'GY')],
      mean,
```

```

    )
  ),
  check.names = FALSE
)

seasonal_means <- aggregate( # season-specific averages
  soybean[c('PH', 'IFP', 'NLP', 'NGP', 'NGL', 'NS', 'MHG', 'GY')],
  by = list(Group = paste('season', soybean$Season)),
  FUN = mean,
)
mean_table <- rbind(overall_means, seasonal_means)
mean_table # combined two tables

```

	Group	PH	IFP	NLP	NGP	NGL	NS	MHG
1	overall	68.38678	15.465	59.08831	135.0858	2.290844	4.071656	168.3223
2	season 1	65.98506	16.485	57.96506	135.2350	2.299688	4.816625	162.2888
3	season 2	70.78850	14.445	60.21156	134.9367	2.282000	3.326687	174.3558
	GY							
1		3418.554						
2		3428.452						
3		3408.655						

e.

Suppose the null hypothesis is the population means of grain yield in two season are the same and the alternative hypothesis is the population means of grain yield in two season are different. Thus

$$H_0 : \mu_1 = \mu_2$$

$$H_1 : \mu_1 \neq \mu_2$$

```

tt_result <- t.test( # Welch Two Sample t-test
  GY ~ Season,
  data = soybean,
  alternative = 'two.sided',
  conf.level = 0.95
)
tt_result

```

Welch Two Sample t-test

data: GY by Season

t = 0.35155, df = 231.85, p-value = 0.7255

alternative hypothesis: true difference in means between group 1 and group 2 is not equal to

95 percent confidence interval:

-91.15712 130.75170

sample estimates:

mean in group 1 mean in group 2

3428.452 3408.655

The t-test produced $t = 0.35155$, $df = 231.85$, $p\text{-value} = 0.7255$. Because the p-value is higher than 0.05, we fail to reject the null hypothesis. The data do not provide sufficient evidence to conclude that the population means of grain yield in two season are the same.

Now try pooled variance

```
ptt_result <- t.test( # pooled Two Sample t-test
  GY ~ Season,
  data = soybean,
  var.equal = TRUE
)
ptt_result
```

Two Sample t-test

data: GY by Season

t = 0.35155, df = 318, p-value = 0.7254

alternative hypothesis: true difference in means between group 1 and group 2 is not equal to

95 percent confidence interval:

-90.99962 130.59420

sample estimates:

mean in group 1 mean in group 2

3428.452 3408.655

Both Welch and pooled t-test led to the same conclusion.

Problem2:

a. and b.

```

adult_name <- c(
  'age',
  'workclass',
  'fnlwgt',
  'education',
  'education_num',
  'marital_status',
  'occupation',
  'relationship',
  'race',
  'sex',
  'capital_gain',
  'capital_loss',
  'hours_per_week',
  'native_country',
  'income'
)
adult <- read.csv('data/adult/adult.data',
  header = FALSE,
  col.names = adult_name,
  na.strings = '?',
  strip.white = TRUE)
# Read the file and rename the columns. This dataset use ? for missing values

head(adult)

```

	age	workclass	fnlwgt	education	education_num	marital_status
1	39	State-gov	77516	Bachelors	13	Never-married
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse
3	38	Private	215646	HS-grad	9	Divorced
4	53	Private	234721	11th	7	Married-civ-spouse
5	28	Private	338409	Bachelors	13	Married-civ-spouse
6	37	Private	284582	Masters	14	Married-civ-spouse

	occupation	relationship	race	sex	capital_gain	capital_loss
1	Adm-clerical	Not-in-family	White	Male	2174	0
2	Exec-managerial	Husband	White	Male	0	0
3	Handlers-cleaners	Not-in-family	White	Male	0	0
4	Handlers-cleaners	Husband	Black	Male	0	0
5	Prof-specialty	Wife	Black	Female	0	0
6	Exec-managerial	Wife	White	Female	0	0

	hours_per_week	native_country	income
1	40	United-States	<=50K

```

2          13 United-States <=50K
3          40 United-States <=50K
4          40 United-States <=50K
5          40          Cuba <=50K
6          40 United-States <=50K

```

```
str(adult)
```

```

'data.frame':  32561 obs. of  15 variables:
 $ age          : int  39 50 38 53 28 37 49 52 31 42 ...
 $ workclass    : chr  "State-gov" "Self-emp-not-inc" "Private" "Private" ...
 $ fnlwgt       : int  77516 83311 215646 234721 338409 284582 160187 209642 45781 159449 .
 $ education    : chr  "Bachelors" "Bachelors" "HS-grad" "11th" ...
 $ education_num : int  13 13 9 7 13 14 5 9 14 13 ...
 $ marital_status: chr  "Never-married" "Married-civ-spouse" "Divorced" "Married-
civ-spouse" ...
 $ occupation   : chr  "Adm-clerical" "Exec-managerial" "Handlers-cleaners" "Handlers-
cleaners" ...
 $ relationship : chr  "Not-in-family" "Husband" "Not-in-family" "Husband" ...
 $ race         : chr  "White" "White" "White" "Black" ...
 $ sex          : chr  "Male" "Male" "Male" "Male" ...
 $ capital_gain : int  2174 0 0 0 0 0 0 0 14084 5178 ...
 $ capital_loss : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hours_per_week: int  40 13 40 40 40 40 16 45 50 40 ...
 $ native_country: chr  "United-States" "United-States" "United-States" "United-
States" ...
 $ income       : chr  "<=50K" "<=50K" "<=50K" "<=50K" ...

```

c.

```
colSums(is.na(adult)) # Count the number of missing values in different columns.
```

	age	workclass	fnlwgt	education	education_num
	0	1836	0	0	0
marital_status	occupation	relationship	race	sex	
	0	1843	0	0	0
capital_gain	capital_loss	hours_per_week	native_country	income	
	0	0	0	583	0

There are 1836 missing values in workclass, 1843 missing values in occupation, and 583 missing values in native_country.

d.

```
num_init <- nrow(adult)
adult_filtered_HPW <- adult[adult$hours_per_week >= 10, ]
# remove hours_per_week >= 10
num_after <- nrow(adult_filtered_HPW)
data.frame(
  stage = c('Before', 'After'),
  sample_size = c(num_init, num_after)
)
```

	stage	sample_size
1	Before	32561
2	After	32103

The sample size reduced from 32561 to 32103.

e.

```
num_init <- nrow(adult_filtered_HPW)
adult_filtered_NA <- adult_filtered_HPW[complete.cases(adult_filtered_HPW), ]
# Remove missing values for occupation, workclass, and native country
num_after <- nrow(adult_filtered_NA)
data.frame(
  stage = c('Before', 'After'),
  sample_size = c(num_init, num_after)
)
```

	stage	sample_size
1	Before	32103
2	After	29873

We removed the observations with missing values in occupation, workclass, or native country because keeping incomplete observations would prevent us from conducting analyses involving these variables. All observations with missing values were removed because the missing values occurred only in occupation, workclass, or native country. The sample size reduced from 32103 to 29873.

f.

```

num_init <- nrow(adult_filtered_NA)
adult_filtered_AGE <- adult_filtered_NA[adult_filtered_NA$age >= 18 &
                                         adult_filtered_NA$age <= 90 &
                                         adult_filtered_NA$education != 'Preschool', ]
# Remove observations with age lower than 18 or higher than 90,
# and educational level is Preschool
num_after <- nrow(adult_filtered_AGE)
data.frame(
  stage = c('Before', 'After'),
  sample_size = c(num_init, num_after)
)

```

	stage	sample_size
1	Before	29873
2	After	29523

The sample size reduced from 29873 to 29523.

g.

```
nrow(adult_filtered_AGE)
```

```
[1] 29523
```

This is the final sample size with 29523.

Problem3:

a.

```

#' Compute the next value in happy number
#'
#' @param x An positive integer.
#'
#' @return An positive integer which is the next happy number of x
#'
#' @examples
#' nextHappy(19)
#' nextHappy(82)

nextHappy <- function(x){
  if(length(x) != 1 ||

```

```

    !is.numeric(x) ||
    is.na(x) ||
    !is.finite(x) ||
    x <= 0 ||
    x != floor(x)){ # make sure x is positive integer and not NA
  stop('Input must be a positive integer.')
}
digits <- as.integer(strsplit(as.character(x), '')[[1]])
# separate the digits in the input
sum(digits ^ 2)
}

```

```
nextHappy(19)
```

```
[1] 82
```

```
nextHappy(82)
```

```
[1] 68
```

b.

```

#' Generate a sequence of happy number
#'
#' @param x A positive integer
#'
#' @return A list with two features:
#' \describe{
#'   \item{sequence}{A sequence of happy number starts from x. For the nth number in the
#'     sequence, the n+1th number is the next happy number of it.}
#'   \item{isHappy}{TRUE if the sequence reaches 1, otherwise 0.}
#' }
#'
#' @examples
#' happySequence(19)
#' happySequence(4)
#'

happySequence <- function(x){
  if(length(x) != 1 ||

```

```

    !is.numeric(x) ||
    is.na(x) ||
    !is.finite(x) ||
    x <= 0 ||
    x != floor(x)){
  # make sure x is positive integer and not NA
  stop('Input must be a positive integer.')
}
if(x == 1){ # if the input is 1, just return the answer
  return(list(sequence = x, isHappy = TRUE))
}
values <- c(x) # sequence of values in happy sequence
while(TRUE){
  x <- nextHappy(x)
  if(x == 1){ # if the value reaches 1, return the answer
    values <- c(values, x)
    return(list(sequence = values, isHappy = TRUE))
  }
  if(x %in% values){
    # if the value repeats and will not reach 1, return the answer
    return(list(sequence = values, isHappy = FALSE))
  }
  values <- c(values, x)
}
}

```

```
happySequence(19)
```

```
$sequence
[1] 19 82 68 100 1
```

```
$isHappy
[1] TRUE
```

```
happySequence(4)
```

```
$sequence
[1] 4 16 37 58 89 145 42 20
```

```
$isHappy
[1] FALSE
```

c.

a.

```
result_HN <- lapply(1: 1000, happySequence)
# calculate happy sequences of each number from 1 to 1000
is_happy <- vapply(result_HN, function(x) x$isHappy, TRUE)
# test if these numbers are happy
sum(is_happy)
```

```
[1] 143
```

There are 143 numbers are happy numbers.

b.

```
sum(is_happy) / 1000 * 100
```

```
[1] 14.3
```

```
# calculate the percentage of happy number
```

The percentage of happy numbers among 1 through 1000 is 14.3%

c.

```
len_happy <- vapply(result_HN, function(x) length(x$sequence), 1)
# give every length of each happy sequence
max(len_happy)
```

```
[1] 19
```

```
which(len_happy == max(len_happy))
```

```
[1] 269 296 629 667 676 692 766 799 926 962 979 997
```

The longest happy sequence before terminating is 19 numbers long, including happy sequences generated by 269, 296, 629, 667, 676, 692, 766, 799, 926, 979, 997